The other fundamental construct that is needed to control the workflow are loops.

Traditionally, programming languages provide at least two kind of loops:

- `for` loops, where we iterate explicitly over some indices or the elements in a container;
- `while` loops, somehow more generic, where a predicate is given and the statements in the loop are executed until the predicate evaluates to true.

There are several variations on both (for instance `do-while`, `for-each` and `repeat-until`) but all loops can always be implemented in terms of the basic while loop.

We will use the for-each loop, and the basic while statement.

For both, we can further control the execution using keywords `continue` (that will force execution to skip to next iteration in the loop) and `break` (that will force execution to immediately exit the loop without checking its predicate).

Providing a few examples should help clarify.

A.4.1 For Loop

To exemplify the use of `for` loops, let's see how to sum all elements in an array `A`: there are at least three ways to do so.

The first one, iterates over the length of the array: